

Smart Energy Management System Using Raspberry Pi Pico W

Abstract—This paper presents a Smart Energy Management and Power Quality Analyzer (PQA) based on the Raspberry Pi Pico W for real-time monitoring of domestic electrical loads.

The proposed system acquires supply voltage and load current through sensor modules, processes the sampled data within the micro-controller, and estimates key quantities such as RMS voltage, RMS current, apparent power, power factor, and energy consumption. In addition to energy measurement, the system evaluates basic power-quality conditions, including voltage sag, swell, fluctuation, overload, and frequency variation, and it provides local indication through an LCD display and status LEDs.

Index Terms—Raspberry PI pico W, Smart energy meter, Hall effect, Wireless notification, IoT Integration

I. INTRODUCTION

Electrical energy monitoring plays a vital role in promoting efficient power usage, reducing wastage, and enabling timely detection of abnormal load conditions in both residential and industrial settings. With the growing adoption of the Internet of Things (IoT), it has become increasingly feasible to build low-cost, compact systems capable of measuring electrical parameters in real time and transmitting this information remotely, without requiring constant physical supervision of the equipment.

This project presents the design and implementation of a real-time AC energy monitoring system built around the Raspberry Pi Pico W, a low-cost microcontroller board with integrated Wi-Fi connectivity. The system employs a ZMPT101B voltage sensor and an ACS712 Hall-effect current sensor to measure the RMS voltage and current of an AC load. From these measurements, the system computes real power and cumulative energy consumption using standard electrical

relationships. The processed readings are displayed locally on a 16×2 I2C LCD for immediate on-site monitoring, while simultaneously being transmitted wirelessly to a remote user via the Telegram Bot API, enabling access to live readings from any internet-connected device.

By integrating analog sensor interfacing, digital signal processing techniques such as RMS estimation, and IoT-based wireless communication on a single resource-constrained embedded platform, this project demonstrates a practical and extensible approach to smart energy monitoring. The system also lays the groundwork for further enhancements such as fault detection, appliance-level load identification, and predictive maintenance through the integration of machine learning techniques, as discussed in the Future Work section of this report.

II. LITERATURE SURVEY

In the recent literatures and projects, a clear shift from conventional metering towards embedded, IoT-enabled monitoring systems for energy management and power-quality assessment has been documented. The focus have further shifted towards power quality analysis and smart monitoring.

1) *An important work by Rakada et al. (2024) reported that harmonics, transients, and voltage fluctuations remain the principal power-quality concerns in IoT-powered smart grids, while interoperability and scalability in practical scenarios limit deployment.*

2) *In a review, Sithole et al. (2024) emphasized that modern energy-management systems increasingly depend on connected sensors, wireless communication, and automated control to im-*

prove monitoring capability, energy efficiency, and operational visibility.:

3) *Implementation-oriented works show how these concepts are realized in practice. Nayak et al. (2023) proposed a low-cost IoT-based power-quality event monitoring system capable of detecting sag, swell, and frequency deviation, with event information transmitted to a control station for corrective action.*:

4) *Madhu et al. (2019) developed a low-cost smart meter for power-quality monitoring in smart-grid infrastructure and used IEEE 1459-2010 processing together with wireless communication to an IoT cloud platform.*:

5) *Silva et al. (2017) presented a monitoring system for electrical energy consumption and power-quality analysis that combined signal conditioning, analog-to-digital conversion, embedded processing, and a real-time graphical interface for viewing scope, harmonics, and data.*:

III. METHODOLOGY

Proposed project performs monitoring of electricity parameters, analysis of the status of power quality, and efficient management of connected loads. The device integrates voltage detection, current detection, signal conditioning, data analysis, displaying, and load control capabilities in one intelligent device. The purpose of the project is to effectively measure electricity usage and abnormalities in the power condition.

A. Components

The proposed projects involves the use of the following components for the analysis and measurement –

1) *Raspberry PI Pico W*: : The Raspberry Pi Pico W is the main controller of the Smart Energy Management and Power Quality Analyzer. It is based on the RP2040, a dual-core Arm Cortex-M0+ processor, and includes built-in Wi-Fi connectivity. GPIO pins have 16-bit ADC for high resolution data input. The Pico W collects voltage and current measurements from the sensors, performs calculations such as power, energy, and power factor, processes the data, and displays information on the LCD. Its wireless capability also enables the integration with IoT platforms for remote monitoring.

2) *ZMPT101B (2mA/ 2mA)*: : The ZMPT101B is a precision AC voltage sensor used to measure the mains supply voltage. It provides electrical isolation between the high-voltage AC line and the low-voltage microcontroller circuit, ensuring safety during operation. The sensor converts the AC voltage into a proportional analog signal that can be read by the Raspberry Pi Pico W for voltage monitoring and power quality analysis.

3) *ACS712 5A*: : The ACS712 is a Hall-effect based current sensor used to measure the current flowing through the load. It can detect both AC and DC currents up to 5A and produces an analog voltage proportional to the measured current. In this project, the sensor helps determine load current, power consumption, and energy usage, enabling efficient energy management.

4) *16 X 2 Blue/White Color Dc 5V Character LCD Display Module with I2C Serial Interface Board Module*:: The 16×2 LCD display is used to show real-time electrical parameters such as voltage, current, power, energy consumption, and system status. The attached I2C interface reduces the number of microcontroller pins required for communication, simplifying wiring and circuit design. It provides a convenient user interface for local monitoring.

B. Working of the Sensors

In the presented project, voltage and current are measured by two separate sensors and then sent to the Raspberry Pi Pico W for processing.

1) *Voltage Sensor (ZMPT101B)*: voltage sensing is performed by the ZMPT101B sensor. It is connected across the AC supply lines to detect the mains voltage. The sensor provides electrical isolation between the high-voltage AC line and the low-voltage controller circuit. It converts the AC mains voltage into a proportional analog signal that the Pico W can read for monitoring and power quality analysis

2) *Current Sensor (ACS712)*: The current sensing is performed by the ACS712 5A sensor. It is placed in series with the load so it can measure the current flowing through that load. The sensor works on the Hall effect. It produces an analog output voltage proportional to the measured current, including AC or DC current up to 5 A.

C. Working Principle

The system operation begins when AC mains power is supplied to the connected load, after which the supply voltage and load current are continuously measured.

Voltage is sensed using the ZMPT101B sensor connected across the AC supply lines, and this sensor provides electrical isolation between the high-voltage line and the low-voltage Raspberry Pi Pico W circuit.

Current is sensed using the ACS712 5A sensor connected in series with the load, where Hall-effect operation generates an analog output proportional to the measured current.

Both analog sensor outputs are connected to the ADC ports of the Raspberry Pi Pico W, which periodically samples the signals and converts them into digital data for programmed mathematical analysis .

D. Micro Python Code

Software: Thonny- 4.1.7 micropython compiler

```
from machine import SoftI2C, Pin, ADC
from pico_i2c_lcd import I2cLcd
import network
import urequests
import ujson
import time
import math
```

```
led = Pin('LED', Pin.OUT)
led.value(1)
time.sleep(1)
```

```

#LCD Display Declaration
LCD_ROWS = 2
LCD_COLS = 16
sda_pin = 0
scl_pin = 1

volt_adc = ADC(Pin(27))
current_adc = ADC(Pin(26))

#ZMPT101B Calibration
SAMPLES = 1000
SAMPLE_INTERVAL_US = 200
CALIBRATION = 190

#ACS712 Calibration
VREF = 3.3
ADC_RESOLUTION = 4095
R_TOP = 967
R_BOTTOM = 1439
DIVIDER_RATIO = R_BOTTOM / (R_TOP + R_BOTTOM)
RAW_SENSITIVITY = 0.185
EFFECTIVE_SENSITIVITY = RAW_SENSITIVITY *
    DIVIDER_RATIO
SAMPLE_WINDOW_MS = 100
NOISE_FLOOR_A = 0.08

#considering pure resistive load
POWER_FACTOR = 1.0
total_energy_wh = 0.0

#WIFI Details
WIFI_SSID = "<WIFI SSID or NAME>"
WIFI_PASSWORD = "WIFI Password"

#Declaration of Telegram Bot
TELEGRAM_BOT_TOKEN = "<BOT Token>"
TELEGRAM_CHAT_ID = "<Chat ID>"
TELEGRAM_SEND_INTERVAL_SEC = 15

#display
def detect_lcd_address(i2c):
    devices = i2c.scan()
    if not devices:
        raise RuntimeError(
            f"No I2C device found on
              SDA={sda_pin}, SCL={scl_pin}
              - check your wiring."
        )
    return devices[0]

#Connection to WIFI
def connect_wifi(ssid, password,
    timeout_sec=15):
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        wlan.connect(ssid, password)
        start = time.ticks_ms()
        while not wlan.isconnected():
            if
                time.ticks_diff(time.ticks_ms(),
                    start) > timeout_sec * 1000:
                    raise RuntimeError("Wi-Fi
                        connection timed out")
            time.sleep_ms(500)

```

```

print("Wi-Fi connected, IP:",
    wlan.ifconfig()[0])
time.sleep(2)
return wlan

#Telegram Message/Notification
def send_telegram_message(text):
    url =
        "https://api.telegram.org/bot{}/sendMessage".format
    payload = {"chat_id": TELEGRAM_CHAT_ID,
        "text": text}
    try:
        response = urequests.post(url,
            json=payload)
        response.close()
    except Exception as e:
        print("Telegram send failed:", e)

#Reading Current RMS
def read_ac_current_rms():
    samples = []
    start_time = time.ticks_ms()
    while time.ticks_diff(time.ticks_ms(),
        start_time) < SAMPLE_WINDOW_MS:
        raw = current_adc.read_u16() >> 4
        voltage = (raw * VREF) /
            ADC_RESOLUTION
        samples.append(voltage)

    if len(samples) == 0:
        return 0.0
    mean_voltage = sum(samples) / len(samples)
    sum_squared = 0.0
    for v in samples:
        centered = v - mean_voltage
        sum_squared += centered * centered
    mean_square = sum_squared / len(samples)
    v_rms = math.sqrt(mean_square)

    i_rms = v_rms / EFFECTIVE_SENSITIVITY
    if i_rms < NOISE_FLOOR_A:
        i_rms = 0.0

    return i_rms

#Reading Voltage RMS
def read_volt_rms():
    samples = []
    for _ in range(SAMPLES):
        samples.append(volt_adc.read_u16() >> 4)
        time.sleep_us(SAMPLE_INTERVAL_US)

    mean = sum(samples) / len(samples)
    sq_sum = sum((s - mean) ** 2 for s in
        samples)
    rms_raw = math.sqrt(sq_sum / len(samples))
    rms_volts_adc = (rms_raw / 4095) * 3.3
    print("raw min:", min(samples), "raw
        max:", max(samples), "raw mean:",
        mean)

    return rms_volts_adc * CALIBRATION

# --- Main ---
connect_wifi(WIFI_SSID, WIFI_PASSWORD)

```

```

time.sleep_ms(100)

i2c = SoftI2C(sda=Pin(sda_pin),
              scl=Pin(scl_pin), freq=500000)
lcd_address = detect_lcd_address(i2c)
print(f"I2C Channel: {i2c}")
print(f"Address: {hex(lcd_address)} | SDA:
      {sda_pin} | SCL: {scl_pin}")

time.sleep_ms(100)

lcd = I2cLcd(i2c, lcd_address, LCD_ROWS,
             LCD_COLS)
lcd.backlight_on()
lcd.clear()
lcd.move_to(0, 0)
lcd.putstr("Initializing...")
lcd.move_to(0, 1)
lcd.putstr("Energy Meter")
time.sleep(2)

lcd.clear()
lcd.move_to(0, 0)
lcd.putstr("Status:")
lcd.move_to(0, 1)
lcd.putstr("Starting...")
time.sleep(1)

print("Starting energy monitor...")
last_time = time.ticks_ms()

last_telegram_time = time.ticks_ms()

while True:
    voltage = read_volt_rms()
    current = read_ac_current_rms()

    power_watts = voltage * current *
        POWER_FACTOR

    now = time.ticks_ms()
    elapsed_hours = (time.ticks_diff(now,
        last_time) ) / (1000 * 3600)
    last_time = now

    total_energy_wh += power_watts *
        elapsed_hours
    total_energy_kwh = total_energy_wh /
        1000.0

    print("V: {:.2f} V | I: {:.2f} A | P:
          {:.2f} W | Energy: {:.2f} kWh".format(
        voltage, current, power_watts,
        total_energy_kwh))

    lcd.clear()
    lcd.move_to(0, 0)
    lcd.putstr("V:{:5.1f}
               I:{:4.2f}A".format(voltage, current))
    lcd.move_to(0, 1)
    lcd.putstr("P:{:5.1f}W
               E:{:5.3f}".format(power_watts,
        total_energy_kwh))

    #Sending the readings through Telegram

```

```

if time.ticks_diff(now,
    last_telegram_time) >=
    TELEGRAM_SEND_INTERVAL_SEC * 1000:
    message = (
        "Energy Monitor Update\n"
        "Voltage: {:.2f} V\n"
        "Current: {:.2f} A\n"
        "Power: {:.2f} W\n"
        "Energy: {:.3f} kWh"
    ).format(voltage, current,
        power_watts, total_energy_kwh)
    send_telegram_message(message)
    print("msg sent")
    last_telegram_time = now

time.sleep(1)

```

E. Algorithm (Pseudo-code) and Block Diagram

```

BEGIN

// ---- Initialization ----
SET led      OUTPUT_PIN(LED); led      HIGH
SET voltAdc  ADC(PIN_26)
SET currAdc  ADC(PIN_27)
SET i2c      I2C(SDA_PIN, SCL_PIN)

wifiConnect(SSID, PASSWORD, TIMEOUT)
IF NOT connected THEN ABORT("Wi-Fi timeout")

lcdAddr      i2cScan(i2c)
IF lcdAddr IS EMPTY THEN ABORT("LCD not
    found")
lcd          initLCD(i2c, lcdAddr)

// ---- Constants ----
V_CAL        <calibration constant>
// from multimeter comparison
I_SENS       0.185
// V/A, ACS712 5A
DIVIDER       R_BOTTOM / (R_TOP + R_BOTTOM)
I_SENS_EFF    I_SENS / DIVIDER
NOISE_FLOOR   0.08
// A
PF            1.0
// unity, resistive load

energyWh      0
lastCycle     now()
lastNotify    now()

//Function      RMS(samples)
FUNCTION RMS(samples[]):
    mean        SUM(samples) / LENGTH(samples)
    sumSquares  0
    FOR EACH s IN samples:
        sumSquares = sumSquares + (s -
            mean)
    RETURN SQRT(sumSquares / LENGTH(samples))

// ---- Main loop ----
LOOP FOREVER

    voltage      RMS( SAMPLE(voltAdc, N) )
                V_CAL
    current      RMS( SAMPLE(currAdc, N) ) /
                I_SENS_EFF

```

```

IF current < NOISE_FLOOR THEN current
0

power      voltage      current      PF

dt_hours      (now() - lastCycle) /
3600000
lastCycle      now()
energyWh      energyWh + power
dt_hours
energyKwh      energyWh / 1000

lcdPrint(voltage, current, power,
energyKwh)
consolePrint(voltage, current, power,
energyKwh)

IF now() - lastNotify
NOTIFY_INTERVAL THEN
msg      format(voltage, current,
power, energyKwh)
telegramSend(msg)
lastNotify      now()
END IF
SLEEP(1000)
END LOOP
END

```

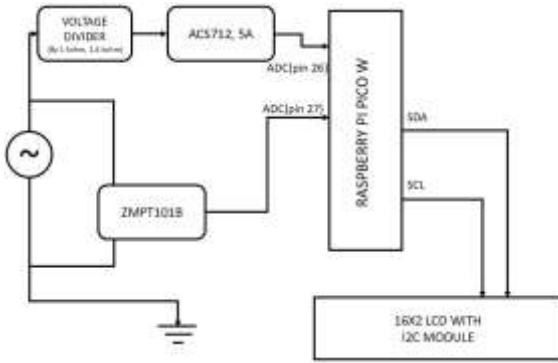


Fig. 1. Block Diagram

F. Wireless Monitoring

To enable remote observation of the system without requiring physical access to the hardware, wireless monitoring was implemented using the Telegram Bot API. The Raspberry Pi Pico W was configured in station mode to connect to a local Wi-Fi network, establishing internet connectivity. A dedicated Telegram bot was created using BotFather, and its unique bot token along with the recipient's chat identifier were used to authenticate and direct messages. At fixed time intervals, the microcontroller formats the measured voltage, current, power, and cumulative energy values into a text payload and transmits it to the Telegram servers via an HTTPS POST request to the bot's sendMessage endpoint, allowing the user to receive live updates on any internet-connected device.

IV. RESULTS AND DISCUSSIONS

A. Thonny : Terminal output Readings

```

MPY: soft reboot
Wi-Fi connected, IP: 192.168.1.2
I2C Channel: SoftI2C(scl=1, sda=0, freq=50000)
Address: 0x27 (SDA) 0 (SCL)
Starting energy monitor...
raw min: 160 raw max: 4095 raw mean: 2766.389
V: 240.94 V | I: 0.31 A | P: 77.95 W | Energy: 0.00 kWh
raw min: 164 raw max: 4095 raw mean: 2749.753
V: 240.57 V | I: 0.31 A | P: 77.50 W | Energy: 0.00 kWh
raw min: 167 raw max: 4095 raw mean: 2743.656
V: 240.44 V | I: 0.31 A | P: 77.94 W | Energy: 0.00 kWh
raw min: 167 raw max: 4095 raw mean: 2829.079
V: 240.55 V | I: 0.31 A | P: 77.52 W | Energy: 0.00 kWh
raw min: 164 raw max: 4095 raw mean: 2761.295
V: 240.49 V | I: 0.31 A | P: 77.89 W | Energy: 0.00 kWh
raw min: 168 raw max: 4095 raw mean: 2756.940
V: 240.43 V | I: 0.31 A | P: 77.30 W | Energy: 0.00 kWh
raw min: 169 raw max: 4095 raw mean: 2767.062
V: 240.55 V | I: 0.31 A | P: 77.22 W | Energy: 0.00 kWh
raw min: 164 raw max: 4095 raw mean: 2751.152
V: 240.54 V | I: 0.31 A | P: 76.65 W | Energy: 0.00 kWh

```

Fig. 2. Thonny Terminal output

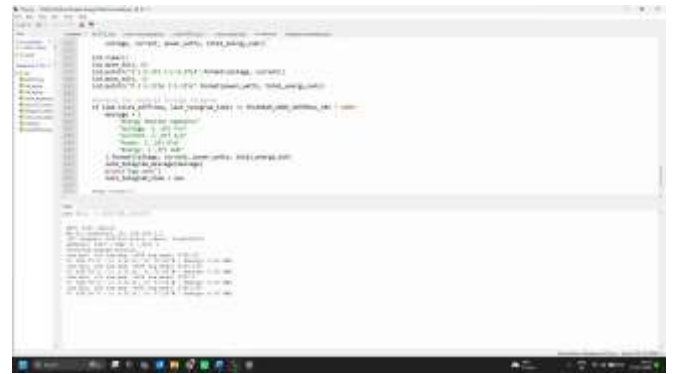


Fig. 3. Thonny output Window

B. LCD



Fig. 4. LCD output: Stage 1



Fig. 5. LCD output: Stage 2



Fig. 6. LCD output: Reading 1



Fig. 7. LCD output: Reading 2

C. Telegram bot Notification



Fig. 8. Telegram output

V. OBSERVATION AND SUMMARY

During testing, the system was powered on and connected to a Wi-Fi network within 7 seconds. The ZMPT101B voltage sensor, after calibration against a multimeter reference, reported mains voltage readings within an accuracy of approximately ± 1.3 percent of the actual measured value. Similarly, the ACS712 current sensor, once calibrated using the measured voltage-divider resistor values, produced current readings that closely matched the expected current drawn by test loads of varying magnitude. At no-load condition, both sensors settled near their expected baseline values, and the noise-floor threshold successfully suppressed spurious non-zero current readings. The computed power and cumulative energy values updated consistently on the LCD display each cycle, and Telegram notifications were received reliably at the configured interval of 15 seconds, confirming stable Wi-Fi connectivity and correct API integration throughout the testing period.

Summary: This project successfully demonstrates a low-cost, real-time AC energy monitoring system built around the Raspberry Pi Pico W microcontroller. By combining a ZMPT101B voltage sensor and an ACS712 current sensor, the system accurately measures RMS voltage and current, from which real power and cumulative energy consumption are derived using standard electrical formulas. The addition of a local LCD display provides immediate on-site feedback, while integration with the Telegram Bot API enables remote, wireless monitoring of the load's electrical parameters from any internet-connected device. The project highlights the practicality of combining analog sensor interfacing, digital signal processing (RMS estimation), and IoT connectivity on a resource-constrained embedded platform, making it a suitable foundation for applications such as home energy monitoring, appliance-level power auditing, or small-scale industrial load tracking.

VI. FUTURE WORK

- **Power Factor Correction for Non-Resistive Loads** — The present system assumes a unity power factor, which is only valid for purely resistive loads. Future iterations could implement synchronized, sample-by-sample multiplication of instantaneous voltage and current to compute true real power, accommodating inductive and capacitive loads such as motors and electronic appliances.
- **Multi-Channel Monitoring** — The system could be expanded to simultaneously monitor multiple loads or circuits using additional ACS712 sensors and multiplexed ADC channels, enabling appliance-level energy breakdown in a household or industrial setting.
- **Data Logging and Cloud Storage** — Instead of only sending periodic Telegram alerts, sensor data could be logged to a cloud database (e.g., Firebase, ThingSpeak, or a custom server) to enable historical trend analysis, usage graphs, and long-term energy consumption reports.

- Overcurrent and Overvoltage Protection — Adding relay-based cutoff mechanisms triggered by threshold-based alerts would allow the system to not only monitor but also protect connected loads from abnormal electrical conditions.
- Mobile Application / Web Dashboard — Rather than relying solely on Telegram messages, a dedicated mobile app or web dashboard could provide richer visualizations, real-time graphs, and remote control features (e.g., turning loads on/off).
- Battery Backup and Low-Power Optimization — Incorporating a battery backup with power-saving sleep modes would make the system resilient to power outages and suitable for remote or off-grid installations.
- Machine Learning-Based Fault Detection - The system can be extended to incorporate machine learning techniques for automated fault detection and predictive maintenance. By continuously logging voltage, current, and power readings, patterns corresponding to normal operating conditions can be established, and deviations from these patterns can be flagged as potential faults

REFERENCES

- [1] Raspberry Pi Foundation, "Raspberry Pi Pico W Datasheet," 2022. [Online]. Available: <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>
- [2] Raspberry Pi Foundation, "Raspberry Pi Pico Python SDK," 2021. [Online]. Available: <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-python-sdk.pdf>
- [3] MicroPython Documentation, "MicroPython for Raspberry Pi Pico." [Online]. Available: <https://docs.micropython.org/en/latest/tp2/quickref.html>
- [4] Allegro MicroSystems, "ACS712: Fully Integrated, Hall-Effect-Based Linear Current Sensor IC," Datasheet, Rev. 09, 2018.
- [5] ZMPT101B Voltage Sensor Module, Manufacturer Datasheet and Application Notes.
- [6] Telegram, "Telegram Bot API Documentation." [Online]. Available: <https://core.telegram.org/bots/api>
- [7] N. Mohan, T. M. Undeland, and W. P. Robbins, Power Electronics: Converters, Applications, and Design, 3rd ed. Hoboken, NJ: Wiley, 2003.
- [8] W. H. Hayt and J. A. Buck, Engineering Electromagnetics, 8th ed. New York: McGraw-Hill, 2012. (for RMS voltage/current fundamentals)
- [9] G. W. Hart, "Nonintrusive appliance load monitoring," Proceedings of the IEEE, vol. 80, no. 12, pp. 1870–1891, Dec. 1992.
- [10] F. Liu, S. Duan, F. Deng, and F. Blaabjerg, "A survey on power quality disturbance detection and fault diagnosis using machine learning techniques," IEEE Access, vol. 8, pp. 213618–213632, 2020.
- [11] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. on Data Mining (ICDM), 2008, pp. 413–422.
- [12] Espressif/MicroPython Community, "urequests Library for MicroPython." [Online]. Available: <https://github.com/micropython/micropython-lib/tree/master/python-ecosys/urequests>